Hide menu

Course Introduction

Introduction to Particle Systems

Video: Project 1: Particle Systems
16 min

Reading: Textbook: Particle Systems
10 min

Video: Adding Forces
9 min

Video: Particle Lifespan
12 min

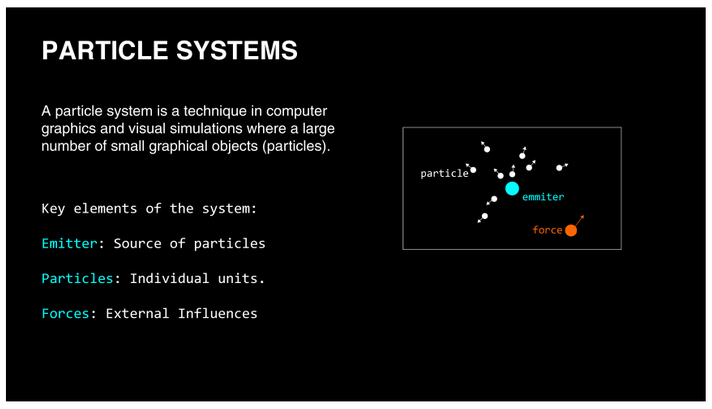
Video: Drawing Particle Trails
15 min

Practice Assignment: Practice Quiz: Particle Systems
30 min

Working with Forces

Textbook: Particle Systems

Creating a particle system is a fascinating way to dive into more advanced graphics programming in Python within Processing. Particle systems are used to simulate fuzzy phenomena like smoke, fire, rain, or even complex moving objects.

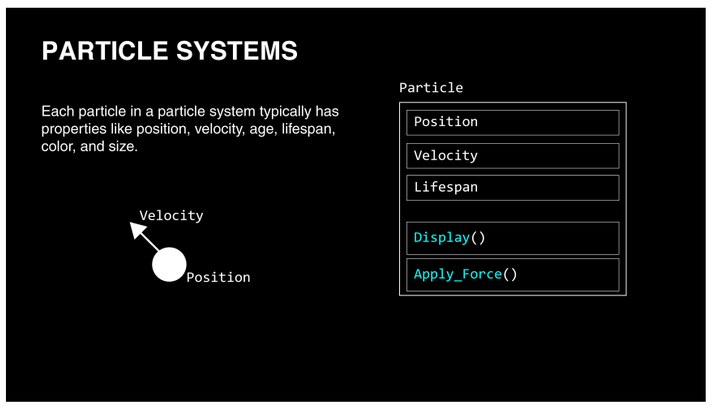


Understanding Particle Systems

A particle system is a collection of many small particles that simulate some collective phenomenon. Each particle is typically a small object with properties like position, velocity, and acceleration.

Basic Steps to Create a Particle System

1. **Define a Particle Class**: Create a class to represent individual particles.
2. **Initialize Particles**: Create instances of particles with initial properties.
3. **Update Particles**: Change properties of each particle over time (like moving them).
4. **Display Particles**: Draw the particles on the canvas.
5. **Remove Dead Particles**: Optionally remove particles that are no longer needed (like those that have faded out).



This will be the foundation of our particle class:

```
import random

class Particle:
    # Particle class constructor
    def __init__(self, pos, world_x, world_y):
        # Initialize position, world dimensions, velocity, acceleration, and counter
        self.pos = pos             # Current position of the particle
        self.world_x = world_x     # Width of the world (or canvas)
        self.world_y = world_y     # Height of the world (or canvas)
        x_vel = random.uniform(-1, 1)  # Random X velocity between -1 and 1
        y_vel = random.uniform(-1, 1)  # Random Y velocity between -1 and 1
        z_vel = random.uniform(-1, 1)  # Random Z velocity between -1 and 1 (for 3D)
        self.vel = PVector(x_vel, y_vel, z_vel)  # Velocity vector
        self.acc = PVector(0, 0, 0)  # Acceleration vector (initialized to 0)

    # Method to update and display the particle each frame
    def run(self):
        self.display()  # Render particle
        self.update()   # Update particle's properties

    # Update particle's position based on its velocity
    def update(self):
        self.pos.add(self.vel)  # Add velocity to position

    # Render the particle on the canvas
    def display(self):
        stroke(255)  # Set stroke color to white
        fill(0, 0, 255)  # Set fill color to blue
        ellipse(self.pos.x, self.pos.y, 5, 5)  # Draw the particle as a small ellipse
```

Here is our base implementation of the particle initialization:

```
import random
from Particle import Particle  # Importing the Particle class

# Canvas dimensions
canvas_size_x = 1200
canvas_size_y = 600

all_particles = []  # List to hold all particle objects

def setup():
    # Create a canvas with the specified width and height
    size(canvas_size_x, canvas_size_y)

    background(0)  # Set the background color to black

    # Create 100 particles
    for i in range(100):
        x = 600  # Set a fixed x-coordinate for all particles
        y = 300  # Set a fixed y-coordinate for all particles
        # Create a new Particle object at (x, y)
        new_particle = Particle(PVector(x, y), canvas_size_x, canvas_size_y)
        all_particles.append(new_particle)  # Add the new particle to the list

def draw():
    background(0)  # Clear the canvas with a black background on each frame

    # Update and draw each particle
    for p in all_particles:
        p.run()  # Call the run method of each particle

```

This basic setup does not yet include forces or other more advanced attributes of particles such as lifespan. We will be covering those in the following lessons.

Go to next item
Completed

Like Dislike Report an issue